

OKDRIVER SENTINEL

SYSTEM ARCHITECTURE

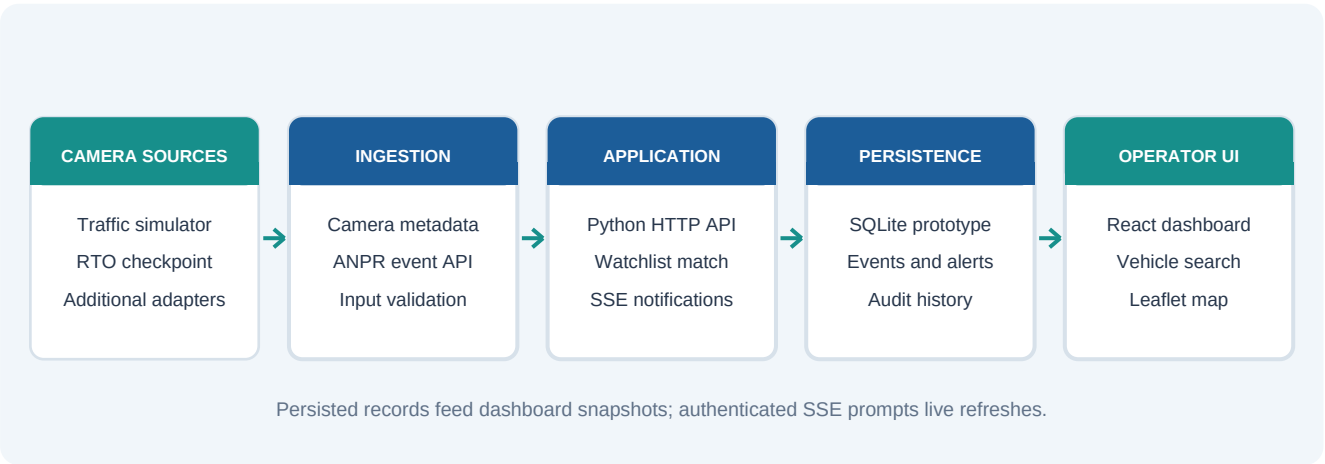
Integrated CCTV monitoring, ANPR event processing, watchlist alerting and vehicle movement search

Architecture note Hiring assignment prototype	Implementation React dashboard + Python HTTP API	Data store SQLite (local prototype)
---	--	---

Purpose and scope

This document describes the working okDriver Sentinel prototype and its path to a production-style deployment. The application brings camera metadata, synthetic ANPR detections, watchlist matches, operator alerts, audit records and map-based vehicle traces into one workflow. It is a demonstration system: camera panels are simulated, endpoint references are metadata, and no real CCTV feed or inference model is connected.

End-to-end architecture



Prototype boundary: the current system runs as one Python process with SQLite and in-memory Server-Sent Events. Its modular interfaces describe where real camera adapters, an inference service, a broker and a production database can be introduced.

1. Components and event flow

The architecture separates source representation, event ingestion, business rules, persistence and operator views. In the prototype these responsibilities run in one API process, while the interface is a React application served from the built frontend.

Component responsibilities

Component	Current responsibility	Production boundary
Camera registry	Stores camera ID, name, center, coordinates, health, protocol, endpoint reference and metadata.	Camera adapters and health workers connect real sources; stream secrets stay outside registry fields.
Source adapters	Simulator entries represent traffic and RTO sources; RTSP/ONVIF/HLS are documented adapter types.	An isolated gateway handles vendor SDKs, RTSP/ONVIF and protocol conversion.
Event API	Validates ANPR payloads, coordinates and confidence; rejects duplicate source IDs.	Use authenticated service identities, quotas, schema versioning and a durable ingest queue.
Correlation	Matches normalized identifiers against active watchlist records and opens an alert.	Move correlation to an idempotent worker consuming durable events.
Persistence	SQLite stores users, centers, cameras, watchlist, detections, alerts and audits.	Use PostgreSQL/MySQL for transactional metadata and object storage for clips.
Realtime delivery	Authenticated SSE informs scoped dashboard clients of relevant changes.	Use broker-backed SSE/WebSocket fan-out with replay cursors and back-pressure.
Operator dashboard	React pages show status, cameras, history, search, alerts, audit and Leaflet traces.	Add pagination, observability, retention controls and accessibility checks.

Event lifecycle

1. Source event	A simulator or adapter produces camera_id, timestamp, vehicle_number, confidence, event type, bounding box and a unique source_id.
2. Validation and deduplication	The API checks required fields, camera existence, confidence bounds and payload shape. A unique source_id prevents duplicate ingestion.
3. Durable write	The detection is saved with its camera, timestamp, owner scope and optional matched watchlist reference.
4. Watchlist correlation	Only an active exact identifier match creates an alert. The alert links to the triggering detection and retains workflow status.
5. Audit and live update	Important actions are recorded. The server publishes a scoped SSE event and connected dashboards reload their authorized snapshot.
6. Search and trace	Operators search detected identifiers; ordered detections with camera coordinates are plotted as numbered sightings on Leaflet.

2. Data model and persistence

The schema is relational and centered on the detection as the event record. Foreign keys and unique constraints enforce important links; indexes support the common vehicle/time, camera/time, owner/time and audit queries.

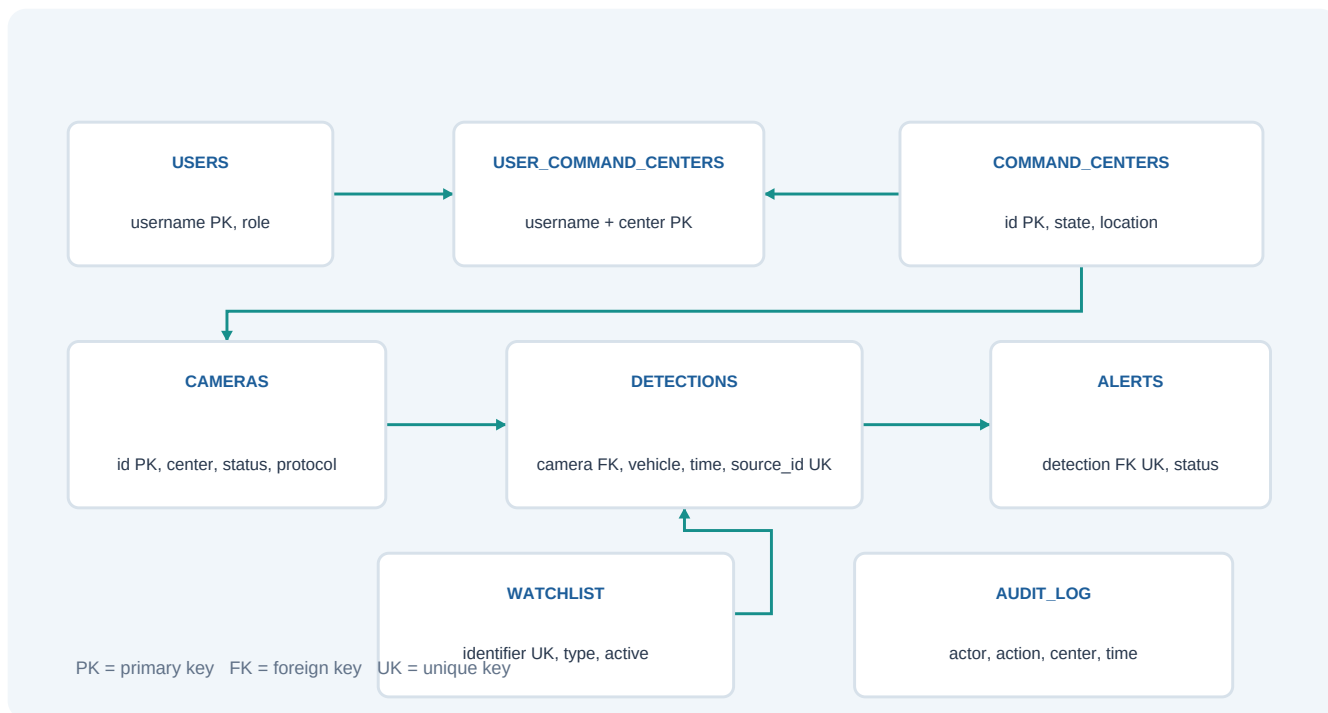


Table	Purpose and key relationships
users	Role-bearing accounts; password material is stored as salted script hashes, not plaintext.
command_centers	City/state locations used to scope camera and operator views.
user_command_centers	Join table for operator-to-command-center membership.
cameras	Camera registry and health metadata; cameras may reference a command center.
watchlist	Unique identifier, entity type, reason, priority and active state. Disabling preserves history.
detections	Normalized ANPR events; camera foreign key, unique source_id, owner, timestamp and optional matched watchlist.
alerts	Workflow record linked uniquely to its triggering detection and watchlist entry; status is Open, Acknowledged or Resolved.
audit_log	Actor, role, command center, action, target, details and timestamp for important operations.

Persistence rules

- SQLite is initialized on server startup and uses foreign-key enforcement. The default local database path is data/okdriver.sqlite3; this file is ignored by Git.
- Additive startup migrations preserve existing assignment data while adding supported ownership and command-center columns.
- Unique source_id and unique alert detection_id constraints provide a database-level duplicate guard.
- Indexes cover detections by vehicle/time and camera/time, cameras by command center, and owner-scoped event/alert history.
- High-volume video bytes do not belong in the relational event database; store clips in object storage and retain references plus metadata.

3. API, identity and operating model

The Python standard-library HTTP server exposes JSON REST endpoints and an authenticated Server-Sent Events stream. OpenAPI details are maintained in `openapi.yaml`. The React client calls the local API; in development Vite proxies `/api` to the Python service.

Main API surfaces

Method and route	Use
POST <code>/api/login</code>	Authenticate a seeded local account and issue an eight-hour bearer token.
GET <code>/api/bootstrap</code>	Load the role- and center-scoped dashboard snapshot.
GET/POST/PATCH <code>/api/cameras</code>	List/filter registry; administrators create and update camera records.
POST <code>/api/events</code>	Validate and persist an ANPR event, correlate watchlist and create an alert on a match.
GET <code>/api/entities</code> and <code>/api/entities/search</code>	List distinct vehicles or retrieve ordered sightings for an exact identifier.
GET/PATCH <code>/api/alerts/{id}</code>	Review and update alert workflow state.
GET/POST/PATCH <code>/api/watchlist</code>	List records; administrators add identifiers or enable/disable a record.
GET <code>/api/events/stream</code>	Receive scoped camera, detection, alert and watchlist change notifications.

Authorization model

- **Administrator:** can view all command centers, manage cameras and watchlist entries, and access full administrative audit activity.
- **Operator:** is restricted to assigned command centers and owner-scoped operational records; may search detections and update alert status. Audit views include administrative/system and the operator's own actions.
- Bearer token is required for protected endpoints. Credentials are configured through environment variables; API inputs are validated before database writes.
- The local service binds to 127.0.0.1 by default. Do not expose it to a network without TLS termination, stronger session controls, rate limiting and secret management.

Local run model

Create a private `.env` from `.env.example`, set unique local admin/operator passwords and a long random signing key, build the frontend with `npm run build`, then run `python server.py`. The server serves the dashboard and API at `http://127.0.0.1:8000`. `.env`, the SQLite data directory and generated frontend build output are excluded from Git.

4. Scale-up, security and known limits

The assignment asks how the prototype can evolve toward approximately 80,000 cameras. That scale is a design target, not a capacity claim for this local build. Workload profiles and load tests are required before sizing infrastructure or estimating costs.

Evolution path

Layer	Prototype	Scale-up direction
Sources and video	Synthetic scenes and simulator camera references.	Regional edge gateways ingest RTSP/ONVIF/vendor sources; relay browser viewing through WebRTC or HLS.
Ingestion and analytics	Synchronous JSON API represents incoming ANPR inference events.	Regional durable broker (Kafka/RabbitMQ) decouples ingest, analytics and alert workers; GPU sizing follows measured FPS and model needs.
Application	Single Python process with in-memory SSE queues.	Stateless API and realtime workers behind a load balancer; broker-backed fan-out, replay and back-pressure.
Data and storage	SQLite metadata/event DB and synthetic sample data.	Managed PostgreSQL/MySQL with partitioning/indexes; object storage with hot/warm/cold clip retention.
Operations	Local process, basic health endpoint and camera heartbeats.	Multi-zone deployment, backups and restore drills, queue-lag metrics, structured logs, tracing, alerts and disaster-recovery objectives.
Network and security	Localhost demo, bearer tokens, demo credentials.	TLS, managed secrets, short-lived scoped identities, network segmentation, rate limits, encryption, access reviews and signed audit trails.

Current limitations

- Camera scenes are simulated. No live RTSP feed, ONVIF discovery, video relay, or AI inference model is included.
- SQLite and in-memory SSE are single-process prototype choices; SSE reconnect does not replay missed events.
- Detection history and distinct-vehicle search need pagination at high data volumes; map tiles and external fonts require internet access.
- There is no production TLS termination, refresh/revocation mechanism, rate limiter, automated test suite, or measured capacity estimate.

Assignment deliverables represented in this repository

The repository includes a working local prototype, setup and limitations in README.md, an architecture overview, an ER model, OpenAPI documentation in openapi.yaml, a synthetic watchlist/camera dataset and a 3-5 minute walkthrough outline. A screen recording and hosted deployment remain candidate-created deliverables.